Programozási nyelvek és paradigmák

Funkcionális programozás

Kozsik Tamás (2020)

Művelettel való paraméterezés

- OOP megközelítés: deferred feature, részleges implementáció
- FP megközelítés: magasabbrendű függvény

Művelettel való paraméterezés OOP stílusban (1)

```
deferred class SORTING[T]
feature
  compare( a, b: T ) : INTEGER deferred end

  sort( ts: attached ARRAY[T] )
    local
      i, n: INTEGER
      item: T
    do
      from n := ts.upper until n <= ts.lower loop
        from i := ts.lower until i = n loop
          if compare(ts[i],ts[i+1]) > 0 then
            item := ts[i]; ts[i] := ts[i+1]; ts[i+1] := item
          end
          i := i+1
        end
        n := n-1
      end
    end -- sort
end -- class SORTING
```

Művelettel való paraméterezés OOP stílusban (2)

```
class SORT_INTEGERS
inherit SORTING[INTEGER]
create make
feature
    compare( i, j: INTEGER ): INTEGER
        do Result := i-j end

    make
        local
            arr: ARRAY[INTEGER]
        do
            arr := <<1,2,3,4,5,4,3,2,1>>
            sort(arr)
            across arr as iter loop
                print(iter.item.out); print(" ")
            end
            print("%N")
        end -- make
end -- class SORT_INTEGERS
```

Eiffel magasabbrendű függvényei: *agent*ekkel

- call agent (parciális alkalmazás)
- inline-agent (lambda)

Call agent

```
agent r                    -- agent Current.r

agent v.r

agent r(12,24)

agent v.r(12,24)

agent v.r(?,24)

agent {MYCLASS}.r(?,?)

PROCEDURE[ MYCLASS, TUPLE[INTEGER,INTEGER] ]
FUNCTION[ MYCLASS, TUPLE[INTEGER,INTEGER], REAL ]
ROUTINE[ MYCLASS, TUPLE[INTEGER,INTEGER] ]
-- MYCLASS és TUPLE elhagyható
```

PROCEDURE

```
feature
    foreach( p: PROCEDURE[TUPLE[REAL]]; a: ARRAY[REAL] )
    do
        across a as i loop
            p.call([i.item])
        end
    end
```

Művelettel való paraméterezés FP stílusban (1)

```
deferred class SORTING[T]
feature
  sort( ts: attached ARRAY[T];
        compare: attached FUNCTION[TUPLE[T,T],INTEGER] )
    local
      i, n: INTEGER
      item: T
    do
      from n := ts.upper until n <= ts.lower loop
        from i := ts.lower until i = n loop
          if compare(ts[i],ts[i+1]) > 0 then
          -- compare.item(ts[i], ts[i+1])
            item := ts[i]; ts[i] := ts[i+1]; ts[i+1] := item
          end
          i := i+1
        end
        n := n-1
      end
    end -- sort
end -- class SORTING
```

Művelettel való paraméterezés FP stílusban (2)

```
class SORT_INTEGERS
inherit SORTING[INTEGER]
create make
feature
    compare( i, j: INTEGER ): INTEGER
        do Result := i-j end

    make
        local
            arr: ARRAY[INTEGER]
        do
            arr := <<1,2,3,4,5,4,3,2,1>>
            sort(arr, agent compare)
            across arr as iter loop
                print(iter.item.out); print(" ")
            end
            print("%N")
        end -- make
end -- class SORT_INTEGERS
```

Inline-agent

```
agent( x, y: INTEGER ): BOOLEAN do Result := x-y > 5 end
```

- Lehet elő- és utófeltétele, lokális változói
- Használhatja a befoglaló osztály feature-jeit
- Nem használhatja a befoglaló rutin (vagy agent) lokális változóit és formális paramétereit
  - explicit átadhatók ezek paraméterként

Művelettel való paraméterezés FP stílusban (3)

```
class SORT_INTEGERS
inherit SORTING[INTEGER]
create make
feature
  make
    local
      arr: ARRAY[INTEGER]
    do
      arr := <<1,2,3,4,5,4,3,2,1>>
      sort(arr, agent( i,j: INTEGER ) : INTEGER
                  do Result := i-j end
      )
      across arr as iter loop
        print(iter.item.out); print(" ")
      end
      print("%N")
    end -- make
end -- class SORT_INTEGERS
```